

DSA COMPLEXITY — FULL FRIDGE NOTES (WITH CODE + EXPLANATION)

Final interview-ready notes — har concept ke saath **code + line-by-line explanation**.

What is Complexity?

Definition

Complexity batati hai ki **input size (n)** badhne par: - ⌚ time kitna badhega - 💾 memory kitni badhegi

🔔 Machine, language, compiler se independent hoti hai.

Types of Complexity

Time Complexity

🌀 Operations ka growth rate

Space Complexity

🌀 Extra memory used (input ko chhod kar)

Why Big-O Notation?

Actual seconds measure nahi karte kyunki har machine different hoti hai.

🔔 **Big-O = worst case growth**

Big-O Notation (Core)

Common Big-O Order

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$$

O(1) — Constant Time

Definition

Input size badhne se time change nahi hota.

Code Example

```
function getFirst(arr) {  
  return arr[0]; // always one operation  
}
```

 **Time:** O(1)

 **Space:** O(1)

O(n) — Linear Time

Definition

Operations input ke proportional badhte hain.

Code Example

```
function printAll(arr) {  
  for (let i = 0; i < arr.length; i++) {  
    console.log(arr[i]); // runs n times  
  }  
}
```

Explanation

- Loop n times chala

 **Time:** O(n)

 **Space:** O(1)

🎓 $O(n^2)$ — Quadratic Time

📌 Definition

Nested loops with same range.

🔗 Code Example

```
function printPairs(arr) {  
  for (let i = 0; i < arr.length; i++) {      // n times  
    for (let j = 0; j < arr.length; j++) {    // n times  
      console.log(arr[i], arr[j]);  
    }  
  }  
}
```

Explanation

- Outer loop $\rightarrow n$
- Inner loop $\rightarrow n$
- Total $\rightarrow n * n$

🕒 Time: $O(n^2)$

💾 Space: $O(1)$

🎩 $O(\log n)$ — Logarithmic Time

📌 Definition

Har step mein problem **half / divide** ho jaati hai.

🔗 Code Example (Binary Search)

```
function binarySearch(arr, target) {  
  let low = 0, high = arr.length - 1;  
  
  while (low <= high) {  
    let mid = Math.floor((low + high) / 2);  
  
    if (arr[mid] === target) return mid;  
    else if (arr[mid] < target) low = mid + 1;  
    else high = mid - 1;  
  }  
}
```

```
    return -1;
}
```

Explanation

- Har step mein search space half ho rahi hai

 **Time:** $O(\log n)$

 **Space:** $O(1)$

$O(n \log n)$ — Linearithmic Time

Definition

Combination of **loop** + **log operation**.

Code Example

```
function example(arr) {
  for (let i = 0; i < arr.length; i++) { // n times
    let x = arr.length;
    while (x > 1) {
      x = x / 2; // log n times
    }
  }
}
```

Explanation

- Outer loop $\rightarrow n$
- Inner while $\rightarrow \log n$

 **Time:** $O(n \log n)$

 **Space:** $O(1)$

Tricky Nested Loop (NOT n^2)

Code Example

```
for (let i = 0; i < n; i++) {  
  for (let j = 1; j < n; j = j * 2) {  
    // work  
  }  
}
```

Explanation

- Outer $\rightarrow n$
- Inner $\rightarrow \log n$

 Time: $O(n \log n)$

Decreasing Work Pattern

Code Example

```
let i = n;  
while (i > 0) {  
  for (let j = 0; j < i; j++) {  
    // work  
  }  
  i = Math.floor(i / 2);  
}
```

Explanation

$$n + n/2 + n/4 + \dots \approx 2n$$

 Time: $O(n)$

 Space: $O(1)$

Space Complexity

Constant Space

```
function sum(a, b) {  
  let c = a + b;  
  return c;  
}
```

 0(1)

Linear Space

```
function createArray(n) {  
  let arr = [];  
  for (let i = 0; i < n; i++) {  
    arr.push(i);  
  }  
  return arr;  
}
```

 0(n)

Recursion Complexity

Single Recursive Call

```
function fun(n) {  
  if (n === 0) return;  
  fun(n - 1);  
}
```

-  Time: 0(n) -  Space: 0(n) (call stack)

Two Recursive Calls

```
function fun(n) {  
  if (n === 0) return;  
  fun(n - 1);  
  fun(n - 1);  
}
```

- ⌚ Time: $O(2^n)$ - 💾 Space: $O(n)$

Best, Average, Worst Case

Linear Search Example

```
function linearSearch(arr, target) {  
  for (let i = 0; i < arr.length; i++) {  
    if (arr[i] === target) return i;  
  }  
  return -1;  
}
```

Case	Time
Best	$O(1)$
Average	$O(n)$
Worst	$O(n)$

Golden Rules (Interview MUST)

- Divide / multiply → $\log n$
- Nested same range → n^2
- Nested different → multiply
- Sequential → add
- Recursion depth → space

Final Verdict

Agar tu in **codes ko dekh ke complexity bol sakta hai**, then:

⌚ **Complexity topic 100% DONE**
